

Raport laboratoryjny - Support Vector Machines

Wstęp do sztucznej inteligencji - Laboratorium 3

Temat: Klasyfikacja danych Breast Cancer Wisconsin z użyciem SVC

Biblioteki: numpy, matplotlib, scikit-learn

Cel raportu

Celem ćwiczenia było przygotowanie klasyfikatora SVM/SVC, przetestowanie wpływu parametru C oraz jądra kernel, wyznaczenie accuracy, macierzy konfuzji, wizualizacji 2D oraz ocena wpływu standaryzacji danych.

Raport został przygotowany w formie wymaganej dla zadania laboratoryjnego: zawiera fragmenty kodu, wyniki, wizualizacje oraz wnioski do kolejnych etapów pracy.

Zadanie 1. Instalacja i import bibliotek

W ćwiczeniu wykorzystano trzy biblioteki wskazane w instrukcji: numpy, matplotlib oraz scikit-learn. Instalacja w środowisku lokalnym lub Colab/Jupyter:

```
pip install numpy matplotlib scikit-learn

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay
```

Wniosek: biblioteki pozwalają wykonać cały proces: wczytanie danych, podział na train/test, skalowanie, trenowanie modelu, ocenę jakości i zapis wykresów.

Zadanie 2-3. Zapoznanie z datasets oraz klasą SVC

Moduł sklearn.datasets zawiera gotowe zbiory danych do nauki i testowania modeli ML. Klasa SVC implementuje klasyfikator Support Vector Classification. Najważniejsze parametry użyte w ćwiczeniu to kernel oraz C.

```
# Przykład konfiguracji klasyfikatora SVC
model = SVC(kernel="linear", C=1, random_state=42)
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
```

Parametr	Znaczenie w ćwiczeniu
kernel="linear"	Jądro liniowe - model tworzy liniową granicę decyzyjną.
kernel="rbf"	Jądro radialne - model może tworzyć nieliniową granicę decyzyjną.
C	Parametr regularyzacji; wpływa na kompromis między szerokim marginesem a błędami klasyfikacji.

Wniosek: do porównania wybrano dwa kernele: linear i rbf, a parametr C testowano dla kilku wartości: 0.01, 0.1, 1, 10, 100.

Zadanie 4a. Wczytanie zbioru Breast Cancer Wisconsin

Dane wczytano funkcją `load_breast_cancer()`. Następnie sprawdzono rozmiar macierzy cech oraz liczbę klas.

```
data = load_breast_cancer()
X = data.data
y = data.target

print("Rozmiar macierzy cech X:", X.shape)
print("Liczba klas:", len(np.unique(y)))
print("Nazwy klas:", data.target_names)
```

Element	Wynik
Rozmiar macierzy cech X	(569, 30)
Liczba klas	2
Nazwy klas	malignant, benign
Liczba cech	30

Wniosek: zbiór zawiera 569 próbek opisanych 30 cechami. Problem jest binarny: klasyfikacja guzów jako malignant albo benign.

Zadanie 4b. Podział danych na zbiór uczący i testowy 70/30

Dane podzielono w proporcji 0.7 / 0.3. Ustawiono `random_state=42`, aby wyniki były powtarzalne, oraz `stratify=y`, aby zachować proporcje klas.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y,
)
```

Zbiór	Liczba próbek	Liczba cech
Uczący	398	30
Testowy	171	30

Wniosek: model uczył się na 398 próbkach, a końcowa ocena accuracy była liczona na osobnym zbiorze testowym zawierającym 171 próbek.

Zadanie 4c. Standaryzacja danych

Standaryzację wykonano poprawnie: scaler został dopasowany wyłącznie do danych uczących, a następnie użyty do transformacji danych uczących i testowych.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit tylko na train
X_test_scaled = scaler.transform(X_test)      # transform na test
```

Wniosek: takie podejście zapobiega data leakage. Dane testowe nie wpływają na dopasowanie skalera, więc ocena modelu pozostaje uczciwa.

Zadanie 4d-4e. Trenowanie SVC i obliczanie accuracy

Przygotowano funkcję pomocniczą, która trenuje model SVC, wyznacza predykcje i oblicza accuracy na zbiorze testowym.

```
def train_and_evaluate(X_train, X_test, y_train, y_test, kernel, C):  
    model = SVC(kernel=kernel, C=C, random_state=42)  
    model.fit(X_train, y_train)  
    y_pred = model.predict(X_test)  
    acc = accuracy_score(y_test, y_pred)  
    return model, y_pred, acc
```

Wniosek: accuracy pozwala szybko porównać konfiguracje modeli, ale w raporcie uzupełniono ją także macierzami konfuzji, żeby sprawdzić rodzaje błędów.

Zadanie 4f. Wpływ parametru C i kernela

Przetestowano wartości C = 0.01, 0.1, 1, 10, 100 dla kernel="linear" i kernel="rbf". Eksperyment wykonano zarówno bez standaryzacji, jak i ze standaryzacją.

```
C_VALUES = [0.01, 0.1, 1, 10, 100]  
KERNELS = ["linear", "rbf"]  
  
results = []  
for preprocessing_name, Xt, Xv in [  
    ("bez standaryzacji", X_train, X_test),  
    ("ze standaryzacja", X_train_scaled, X_test_scaled),  
]:  
    for kernel in KERNELS:  
        for C in C_VALUES:  
            _, y_pred, acc = train_and_evaluate(Xt, Xv, y_train, y_test, kernel, C)  
            results.append({  
                "preprocessing": preprocessing_name,  
                "kernel": kernel,  
                "C": C,  
                "accuracy": acc,  
            })
```

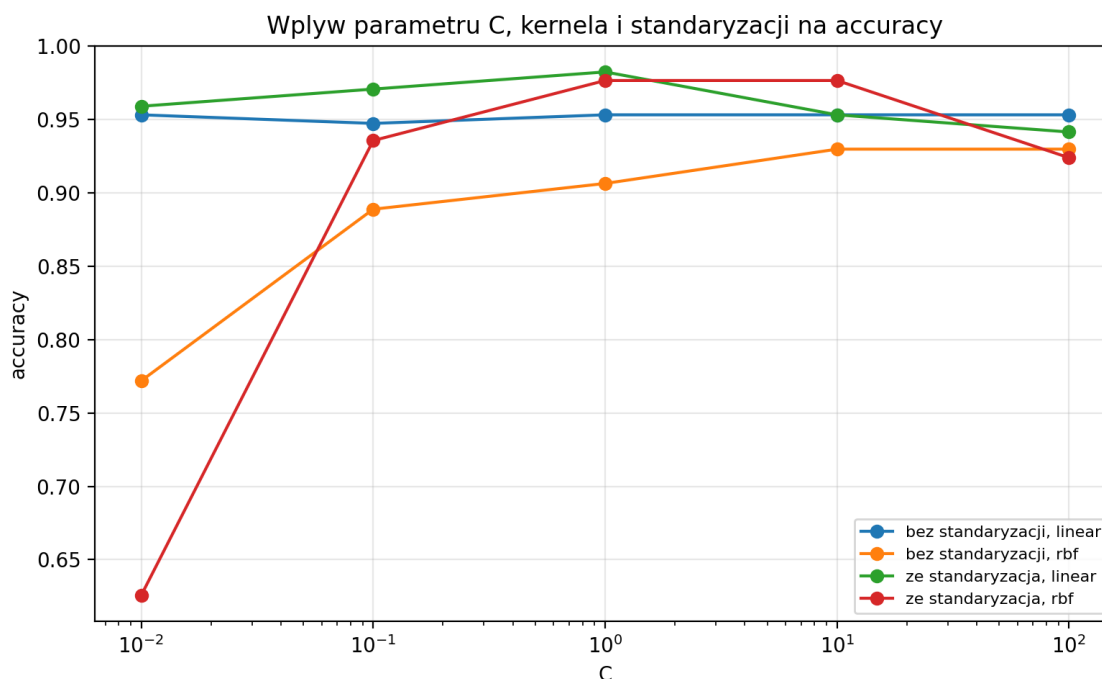
Tabela wyników accuracy:

Preprocessing	Kernel	C	Accuracy
bez standaryzacji	linear	0.01	0.9532
bez standaryzacji	linear	0.1	0.9474
bez standaryzacji	linear	1	0.9532
bez standaryzacji	linear	10	0.9532
bez standaryzacji	linear	100	0.9532
bez standaryzacji	rbf	0.01	0.7719
bez standaryzacji	rbf	0.1	0.8889
bez standaryzacji	rbf	1	0.9064
bez standaryzacji	rbf	10	0.9298
bez standaryzacji	rbf	100	0.9298
ze standaryzacja	linear	0.01	0.9591
ze standaryzacja	linear	0.1	0.9708
ze standaryzacja	linear	1	0.9825
ze standaryzacja	linear	10	0.9532
ze standaryzacja	linear	100	0.9415
ze standaryzacja	rbf	0.01	0.6257

Preprocessing	Kernel	C	Accuracy
ze standaryzacja	rbf	0.1	0.9357
ze standaryzacja	rbf	1	0.9766
ze standaryzacja	rbf	10	0.9766
ze standaryzacja	rbf	100	0.9240

Wniosek: najlepszy wynik uzyskał model linear po standaryzacji dla C=1: accuracy=0.9825. Model rbf po standaryzacji dla C=1 oraz C=10 uzyskał accuracy=0.9766.

Wizualizacja wpływu C, kernela i standaryzacji



Rys. 1. Porównanie accuracy dla różnych wartości C, kerneli i wariantów preprocessingowych.

Wniosek: zbyt małe lub zbyt duże C może pogorszyć wynik. W tym eksperymencie C=1 było najkorzystniejsze dla kernela linear, a dla rbf bardzo dobre wyniki dały C=1 i C=10.

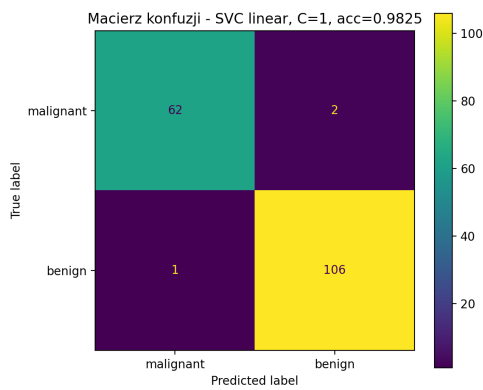
Zadanie 4g. Najlepsze konfiguracje i macierze konfuzji

Na podstawie accuracy wybrano najlepszą konfigurację dla każdego kernela na danych po standaryzacji.

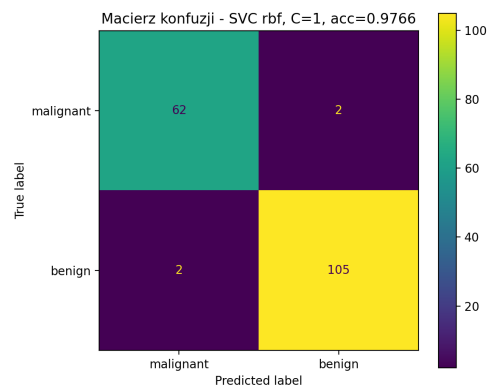
```
scaled_results = [r for r in results if r["preprocessing"] == "ze standaryzacja"]
best_by_kernel = {}
for kernel in KERNELS:
    kernel_results = [r for r in scaled_results if r["kernel"] == kernel]
    best_by_kernel[kernel] = max(kernel_results, key=lambda r: r["accuracy"])
```

Kernel	Najlepsze C	Accuracy	Macierz konfuzji
linear	1	0.9825	[[62, 2], [1, 106]]
rbf	1	0.9766	[[62, 2], [2, 105]]

Wniosek: model linear błędnie sklasyfikował 3 próbki testowe, a model rbf 4 próbki testowe. Oba modele osiągnęły bardzo wysoką skuteczność, ale linear był minimalnie lepszy.



Rys. 2. Macierz konfuzji - linear, C=1.



Rys. 3. Macierz konfuzji - rbf, C=1.

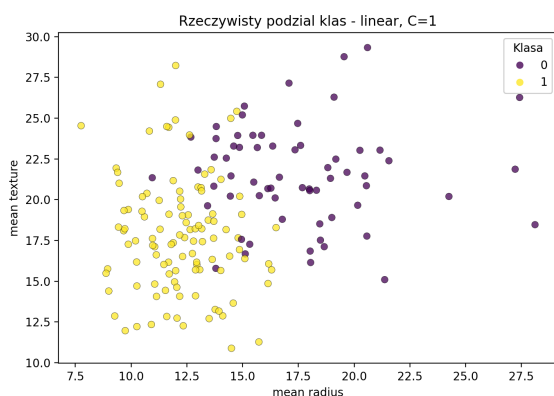
Zadanie 4g. Wizualizacja 2D danych testowych

Do wizualizacji wybrano dwie cechy: mean radius oraz mean texture. Modele były trenowane na wszystkich 30 cechach, a wykres 2D służy wyłącznie do wizualnego porównania klas rzeczywistych i przewidzianych.

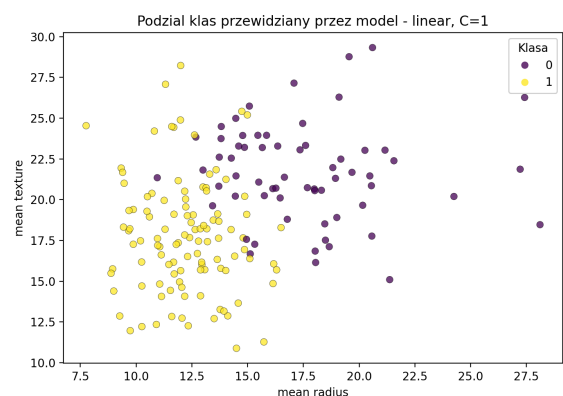
```
feature_idx = (0, 1) # mean radius, mean texture

save_scatter_plot(
    X_test,
    y_test,
    feature_names,
    feature_idx,
    "Rzeczywisty podział klas - linear, C=1",
    "scatter_true_linear.png",
)

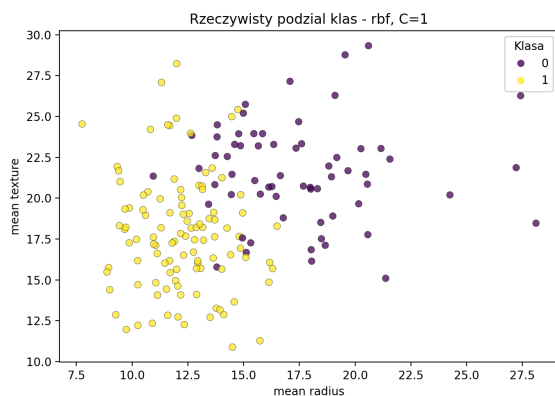
save_scatter_plot(
    X_test,
    y_pred,
    feature_names,
    feature_idx,
    "Podział klas przewidziany przez model - linear, C=1",
    "scatter_pred_linear.png",
)
```



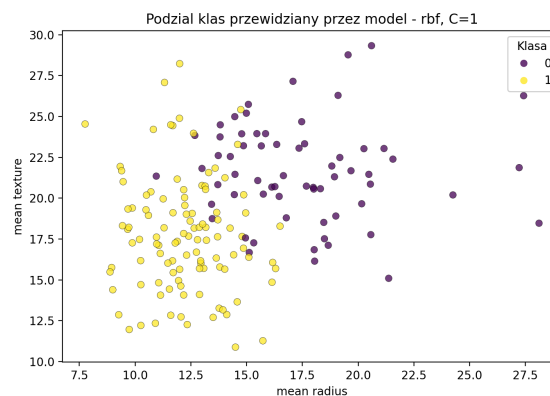
Rys. 4. Klasy rzeczywiste - linear.



Rys. 5. Klasy przewidziane - linear.



Rys. 6. Klasy rzeczywiste - rbf.



Rys. 7. Klasy przewidziane - rbf.

Wniosek: na wykresach widać, że klasy częściowo nakładają się w przestrzeni wybranych dwóch cech, dlatego pojedyncza wizualizacja 2D nie oddaje całej informacji wykorzystanej przez model. SVC korzystał ze wszystkich 30 cech.

Zadanie 4h. Analiza wpływu standaryzacji

Porównano wyniki modeli trenowanych bez standaryzacji oraz po standaryzacji danych. Poniżej pokazano najważniejsze różnice dla $C=1$.

Kernel	C	Accuracy bez standaryzacji	Accuracy ze standaryzacją	Różnica
linear	1	0.9532	0.9825	+0.0293
rbf	1	0.9064	0.9766	+0.0702

Standaryzacja jest istotna w metodach SVM, ponieważ algorytm bazuje na odległościach, iloczynach skalarnych oraz funkcjach jądra. Jeżeli cechy mają różne zakresy wartości, cechy o największej skali mogą dominować proces uczenia, a model może gorzej generalizować.

Wniosek: standaryzacja poprawiła wynik dla obu kerneli. Poprawa była szczególnie widoczna dla kernela rbf: accuracy wzrosło z 0.9064 do 0.9766 dla $C=1$.

Podsumowanie końcowe

W ćwiczeniu wykonano wszystkie wymagane etapy: zainstalowano i wykorzystano biblioteki, wczytano zbiór Breast Cancer Wisconsin, podzielono dane na train/test, wykonano standaryzację, wytrenowano modele SVC, porównano wartości C i kerneli linear/rbf, wyznaczono accuracy, macierze konfuzji oraz wykresy 2D.

Najważniejszy element	Wynik / interpretacja
Najlepszy model	SVC(kernel="linear", $C=1$) po standaryzacji
Najlepsze accuracy	0.9825
Najlepszy rbf	SVC(kernel="rbf", $C=1$), accuracy=0.9766
Wpływ C	C wpływa na kompromis między szerokością marginesu a karą za błędy.
Wpływ kernela	linear był minimalnie lepszy w tym eksperymencie, rbf również działał bardzo dobrze.
Wpływ standaryzacji	Standaryzacja poprawiła wyniki, szczególnie dla rbf.